

Пояснительная записка

Эмулятор LoRa mesh-сети на базе Meshtastic-прошивки

Перечень сокращений и условных обозначений

Аббревиатура	Расшифровка
LoRa	Long Range — технология радиосвязи дальнего радиуса
CSS	Chirp Spread Spectrum — расширение спектра методом чирп-сигналов
LFM	Линейная частотная модуляция
SF	Spreading Factor — коэффициент расширения спектра
BW	Bandwidth — ширина полосы пропускания
CR	Code Rate — скорость помехоустойчивого кодирования
LDRO	Low Data Rate Optimization — оптимизация низкой скорости передачи
AWGN	Additive White Gaussian Noise — аддитивный белый гауссовский шум
SDR	Software Defined Radio — программно-определяемое радио
SITL	Software-in-the-Loop — программная эмуляция в замкнутом контуре
HITL	Hardware-in-the-Loop — аппаратная эмуляция в замкнутом контуре
LoRaWAN	LoRa Wide Area Network — протокол сетевого уровня поверх LoRa
ADR	Adaptive Data Rate — адаптивная скорость передачи данных
OTAA	Over-The-Air Activation — активация по воздуху
ABP	Activation By Personalization — статическая активация
NetID	Network Identifier — идентификатор сети LoRaWAN
DevAddr	Device Address — адрес конечного устройства
KISS	Keep It Simple, Stupid — протокол инкапсуляции пакетов для TNC
AES	Advanced Encryption Standard — стандарт симметричного шифрования
MVC	Model-View-Controller — архитектурный паттерн
FFT	Fast Fourier Transform — быстрое преобразование Фурье
IQ	In-phase / Quadrature — синфазная/квадратурная составляющая сигнала
CRC	Cyclic Redundancy Check — циклический избыточный код
GUI	Graphical User Interface — графический интерфейс пользователя

Введение

В современных условиях возрастающей зависимости от централизованных систем связи (сотовых сетей, интернета) особую актуальность приобретают децентрализованные системы связи. Одним из востребованных решений является технология Meshtastic — открытый проект, реализующий децентрализованную mesh-сеть для приемо-передатчиков LoRa.

Технология LoRa (Long Range) использует запатентованную модуляцию с расширением спектра методом линейной частотной модуляции. Данная модуляция обеспечивает высокую помехоустойчивость и дальность передачи при минимальном энергопотреблении, что делает её основой большинства LPWAN-решений. LoRaWAN является стандартным протоколом сетевого уровня поверх LoRa, а Meshtastic реализует поверх LoRa децентрализованную mesh-маршрутизацию.

При разработке распределённых беспроводных систем, таких как mesh-сети, ключевой задачей является корректная оценка поведения радиоканала. Экспериментальная проверка таких систем в реальных условиях требует значительных материальных и временных ресурсов: необходимо несколько физических устройств, возможность их размещения на требуемых расстояниях, а повторяемость экспериментов существенно ограничена. В связи с этим особую актуальность приобретают программные методы эмуляции физического уровня радиоканала.

Одним из подходов является концепция Software-in-the-Loop (SITL), при которой встроенное программное обеспечение и алгоритмы выполняются на хост-компьютере путём запуска в виртуальной среде. В отличие от Hardware-in-the-Loop (HITL), где используются реальные физические компоненты, SITL позволяет эмулировать все уровни программно, что снижает затраты и обеспечивает высокую воспроизводимость экспериментов.

Настоящая работа посвящена разработке программного эмулятора LoRa mesh-сети на базе реальной прошивки Meshtastic с применением подхода SITL. Документ включает описание физического уровня LoRa, протоколов LoRaWAN и Meshtastic, архитектуры разработанного программного обеспечения и результатов его работы.

1. Постановка задачи

1.1. Определение задач

Цель работы — разработка программного SITL-эмулятора полного стека Meshtastic, позволяющего эмулировать поведение сети на всех уровнях без использования физических устройств.

Для достижения цели были поставлены следующие задачи:

1. Разработать модель канала связи, реализующую распространение IQ-сигналов между виртуальными узлами с учётом затухания и шума.
2. Реализовать описание узлов в виде двух независимых графов обработки сигналов (путь приёма и путь передачи) на базе фреймворка FutureSDR.
3. Разработать механизм, связывающий узлы FutureSDR с моделью канала связи (блоки Signal Subscriber и Signal Publisher через каналы crossbeam).
4. Реализовать драйвер эмуляции физического интерфейса Meshtastic (KISS over UDP).
5. Реализовать адаптацию прошивки Meshtastic для работы в виртуальной среде (платформа `native_virtual`).

6. Реализовать базовый графический интерфейс для управления виртуальными узлами и конфигурацией сети.

Ограничения и допущения:

- Эмуляция выполняется на одном хост-компьютере; реальная радиопередача не используется.
- Модель затухания — свободное пространство (d^{-3}); многолучевое распространение не моделируется.
- Поддерживаются регионы EU433, EU868, RU в соответствии со спецификацией Meshtastic.
- Изоляция каждого виртуального узла обеспечивается Docker-контейнером.

2. Обзорная часть

2.1. Физический уровень LoRa

Среди множества технологий сетей дальнего радиуса действия с низким энергопотреблением LoRaWAN стала одним из самых популярных и широко распространённых решений. LoRa является физическим уровнем технологии маломощных сетей дальнего радиуса (LPWAN).

LoRa использует запатентованную модуляцию с расширением спектра методом линейной частотной модуляции (или расширение спектра с помощью чирпов). Данная модуляция используется вместе с помехоустойчивым кодированием и перемежением.

Протокол LoRaWAN является открытым стандартом, но физический уровень, называемый LoRa, запатентован (Seller & Sornin U.S. Patent 9 252 834) и является собственностью компании, что послужило причиной обратного проектирования для понимания структуры кадров, кодирования и модуляции.

2.1.1. Линейная частотная модуляция (LFM/CSS)

LoRa использует модуляцию, которая определяется двумя параметрами передачи: коэффициентом расширения спектра (SF) и шириной пропускания (BW).

Каждый символ состоит из $N = 2^{SF}$ чипов. Каждому значению символа s соответствует определённый циклический сдвиг начальной частоты, где $s \in \{0, \dots, N\}$. Время передачи одного символа:

$$T_s = N / BW$$

Символ кодируется как chirp-сигнал, в котором частота линейно нарастает от $-BW/2$ до $+BW/2$ за время T_s , после чего циклически повторяется. Циклический сдвиг, соответствующий значению символа s , определяет начальную частоту chirp-а. На приёмной стороне для демодуляции применяется операция умножения принятого сигнала на down-chirp с последующим FFT — пик FFT соответствует номеру символа.

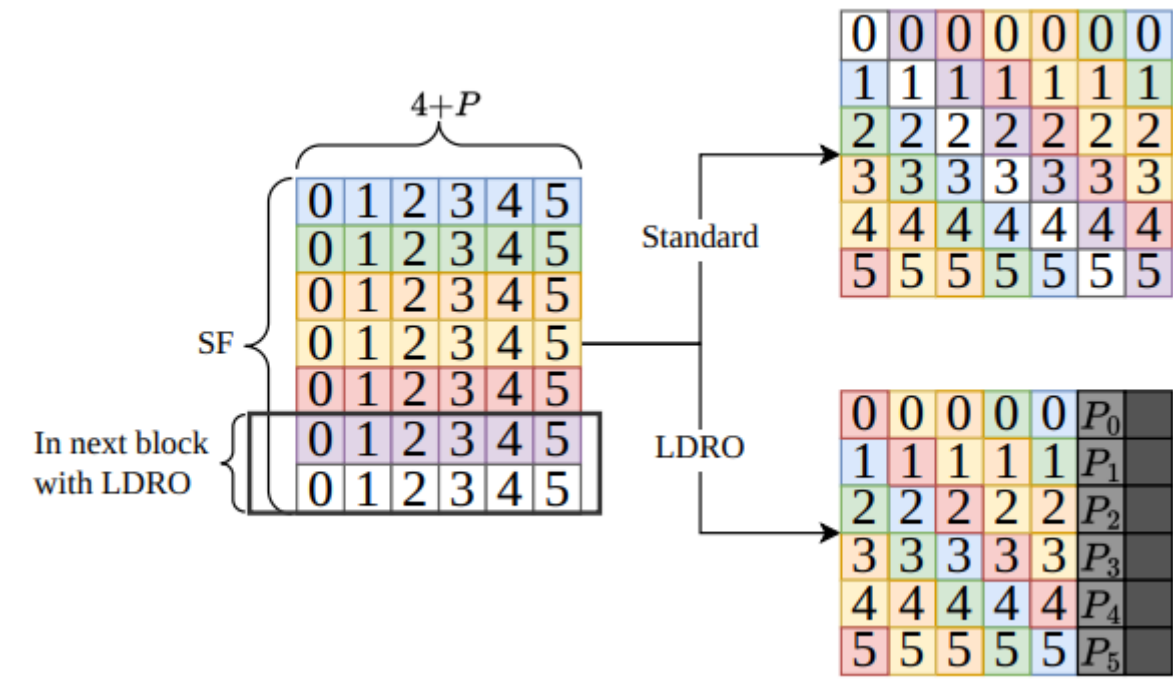


Рисунок 2.1 — Спектрограмма с границами символов LoRa

Ключевые свойства CSS-модуляции:

- **Устойчивость к доплеровскому сдвигу:** частотный сдвиг сигнала проявляется как временной сдвиг в демодуляторе, который компенсируется алгоритмом синхронизации.
- **Устойчивость к многолучевому распространению:** расширение спектра снижает влияние интерференции от отражений.
- **Ортогональность SF:** символы с разными SF не интерферируют друг с другом даже на одной частоте.

2.1.2. Структура кадра и параметры LoRa

Существуют два вида фрейма — explicit и implicit. Отличие в наличии заголовка, который присутствует только в explicit-кадрах. Через заголовок кадр задаёт скорость помехоустойчивого кодирования (CR 4/5, 4/6, 4/7, 4/8) и наличие 2 байт контрольной суммы (CRC) в конце кадра.

Кадр состоит из преамбулы, заголовка (только для explicit), полезной нагрузки и контрольной суммы.

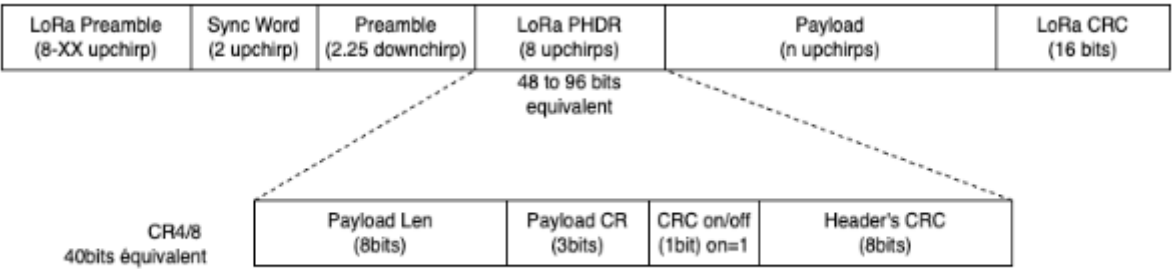


Рисунок 2.2 — Структура кадра LoRa

Процесс формирования кадра включает следующие этапы (рисунок 2.3):

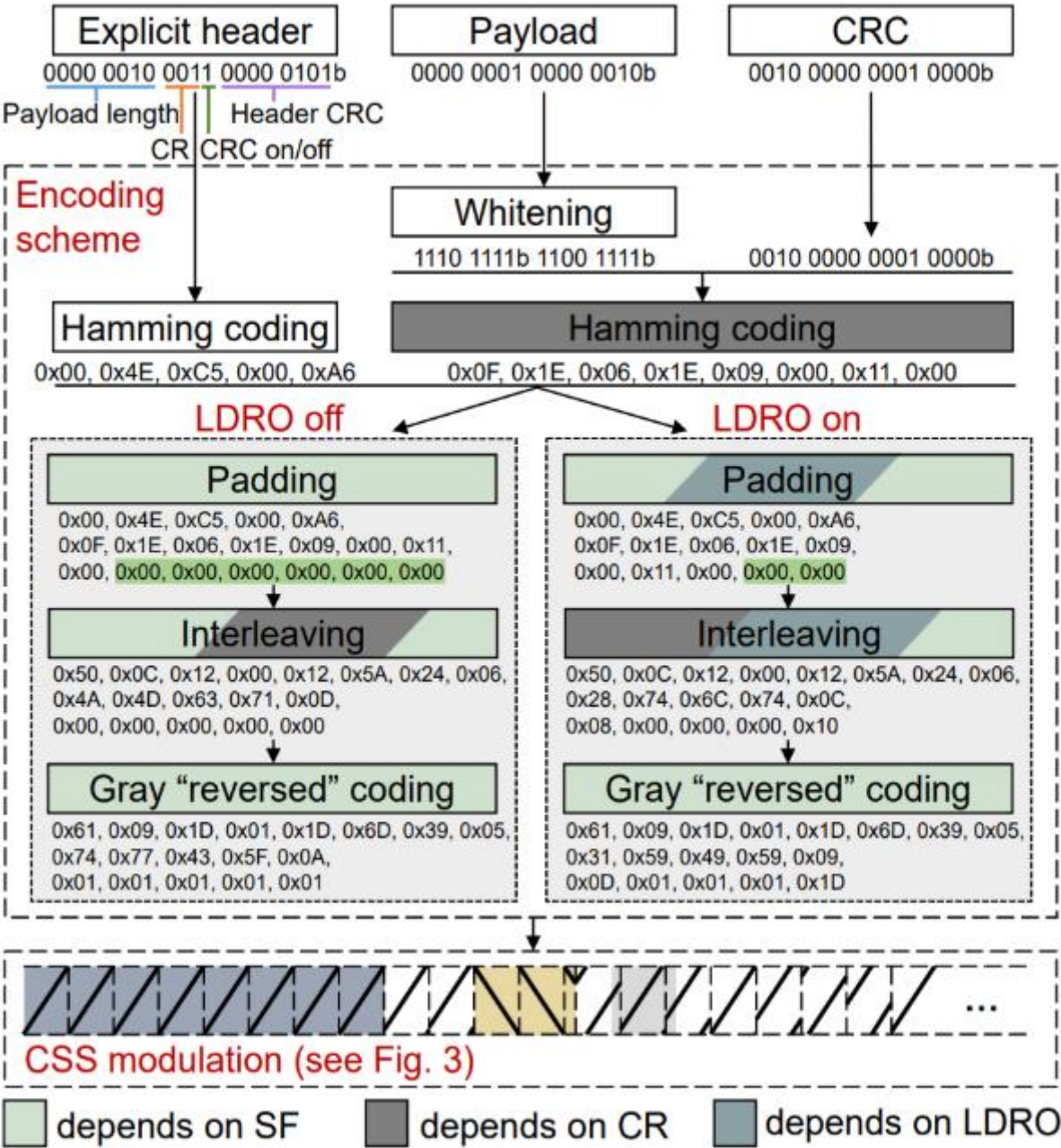


Рисунок 2.3 — Структура кодирования, взбалтывания и перемежения

Взбалтывание (Whitening)

Операция взбалтывания рандомизирует битовые последовательности и уменьшает вероятность появления длинных серий единиц или нулей на основе операции суммы по модулю 2, для предотвращения проблем при передаче данных. Взбалтывается только полезная нагрузка.

Псевдослучайная последовательность сгенерирована регистром сдвига с линейной обратной связью (LFSR). Полином обратной связи:

$$x^8 + x^6 + x^5 + x^4 + 1$$

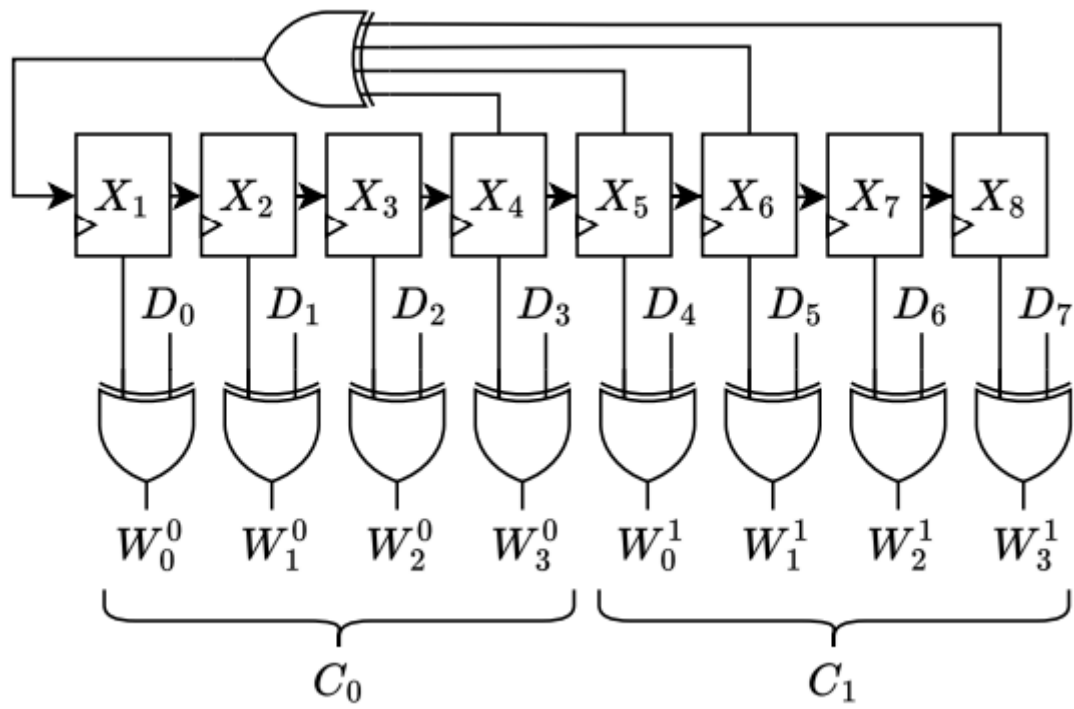


Рисунок 2.4 — Функциональная схема взбалтывания для одного байта данных

Заголовок

Заголовок используется для передачи информации, необходимой для декодирования кадра. Состоит из длины полезной нагрузки, наличия CRC для полезной нагрузки, скорости кодирования и контрольной суммы для заголовка.

CRC заголовка вычисляется по формуле:

$$H_CRC = G \cdot \vec{h}$$

где $\vec{h} = [L_7, L_6, \dots, C]$ является вектором битов, G — генераторная матрица (5×11).

L_4	L_5	L_6	L_7
L_0	L_1	L_2	L_3
C	P_0	P_1	P_2
H_4			
H_0	H_1	H_2	H_3

Рисунок 2.5 — Структура заголовка кадра LoRa

Параметры модуляции LoRa

- **SF (Spreading Factor, 7–12)** — коэффициент расширения спектра. Определяет количество чипов на символ ($N = 2^{SF}$). Увеличение SF повышает дальность и чувствительность, но снижает

скорость передачи данных. SF7 обеспечивает наибольшую скорость (~5,47 кбит/с при BW=125 кГц), SF12 — наибольшую дальность (~0,29 кбит/с).

- **BW (Bandwidth)** — ширина полосы пропускания. Поддерживаемые значения: 62,5; 125; 250; 500 кГц. Увеличение BW повышает скорость передачи, но снижает чувствительность приёмника (~6 дБ на удвоение полосы).
- **CR (Code Rate, 4/5–4/8)** — скорость помехоустойчивого кодирования кодом Хэмминга. CR 4/5 добавляет минимальную избыточность (1 бит на 4), CR 4/8 — максимальную (4 бита на 4). Увеличение избыточности повышает устойчивость к ошибкам за счёт снижения скорости.
- **LDRO (Low Data Rate Optimization)** — режим оптимизации для низких скоростей передачи. Активируется при большом SF и узкой BW для компенсации временного дрейфа частоты передатчика. Уменьшает размер блока перемежителя с $SF \cdot (4+P)$ до $(SF-2) \cdot (4+P)$.

Перемежение (Interleaving)

Перемежение используется для распределения битов кодового слова с целью улучшения возможности исправления ошибок кода. Перемежитель берёт двоичную матрицу D размером $SF \cdot (4+P)$, в которой каждая строка является словом.

Диагональное перемежение определяется уравнением:

$$I_{\{i,j\}} = D_{\{[i-(j+1)] \bmod SF, i\}}$$

где $i \in \{0, \dots, 3+P\}$ и $j \in \{0, \dots, SF-1\}$

$$G = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Рисунок 2.6 — Иллюстрация перемежения (стандартного и с LDRO)

Преобразование в код Грея

Каждая строка перемеженной матрицы интерпретируется как двоичное значение. Операция преобразования в код Грея позволяет соседним символам отличаться только одним битом, что уменьшает влияние одиночных ошибок демодуляции.

Преобразование, используемое LoRa, задаётся формулой:

$$s = [I_i \oplus (I_i \gg 1) + 1] \bmod SF$$

где I_i — i-я строка матрицы.

Преамбула

Для синхронизации кадр LoRa начинается с преамбулы. Преамбула содержит N_{up} восходящих чирпов, затем 2 символа сетевого идентификатора (sync word) и в конце 2,25 нисходящих чирпа.

Число $N_{up} \in \{0, \dots, 65535\}$, где 8 является наиболее распространённым. Типовое значение сетевого идентификатора: 0x34 для LoRaWAN или 0x12 для частных сетей. В Meshtastic используется значение 0x2B (для частной сети).

Разбивка sync word на символы S_{w0} и S_{w1} :

$$S_{w0} = (S_w \gg 4) * 8 \quad S_{w1} = (S_w \& 0x0F) * 8$$

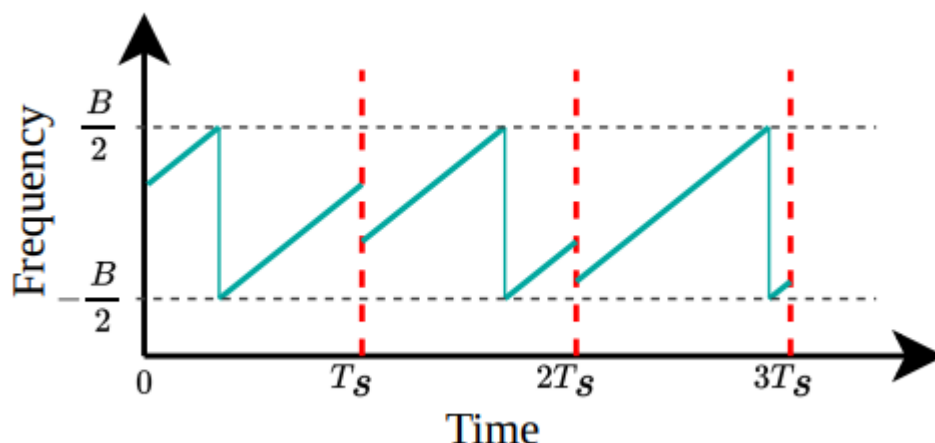


Рисунок 2.7 — Спектрограмма преамбулы кадра LoRa

2.2. LoRaWAN

LoRaWAN — это MAC-протокол, ориентированный на передачу данных на большие расстояния при минимальном энергопотреблении. Описывает организацию сети поверх физического уровня LoRa.

2.2.1. Модель сети

Основные компоненты сети LoRaWAN:

- **Конечное устройство (End-device)** — датчик, исполнительный механизм, либо устройство, совмещающее оба типа. Обмен данными происходит через радиошлюз.
- **Радиошлюз (Radio Gateway)** — работает полностью на физическом уровне:
 - Uplink (восходящий канал) — декодирует принятые радиопакеты и передаёт их на сетевой сервер без обработки.
 - Downlink (нисходящий канал) — запросы от сетевого сервера передаются конечному устройству.
- **Сетевой сервер (Network Server)** — центральный управляющий элемент LoRaWAN сети. Каждый сетевой сервер имеет один или более ID (IEEE EUI64). Выполняет функции:
 - Проверки адреса конечного устройства и кадров.
 - Механизм подтверждения (ACK).

- Адаптация скорости передачи данных (ADR).
 - Отправка ответов на MAC-запросы.
 - Пересылка сообщений uplink на сервер приложений.
- **Сервер активации (Join Server)** — отвечает за процедуру активации по воздуху (OTAA). Генерирует сессионные ключи (Network Session Key, Application Session Key).
 - **Сервер приложений (Application Server)** — обрабатывает все данные, полученные от конечного устройства.

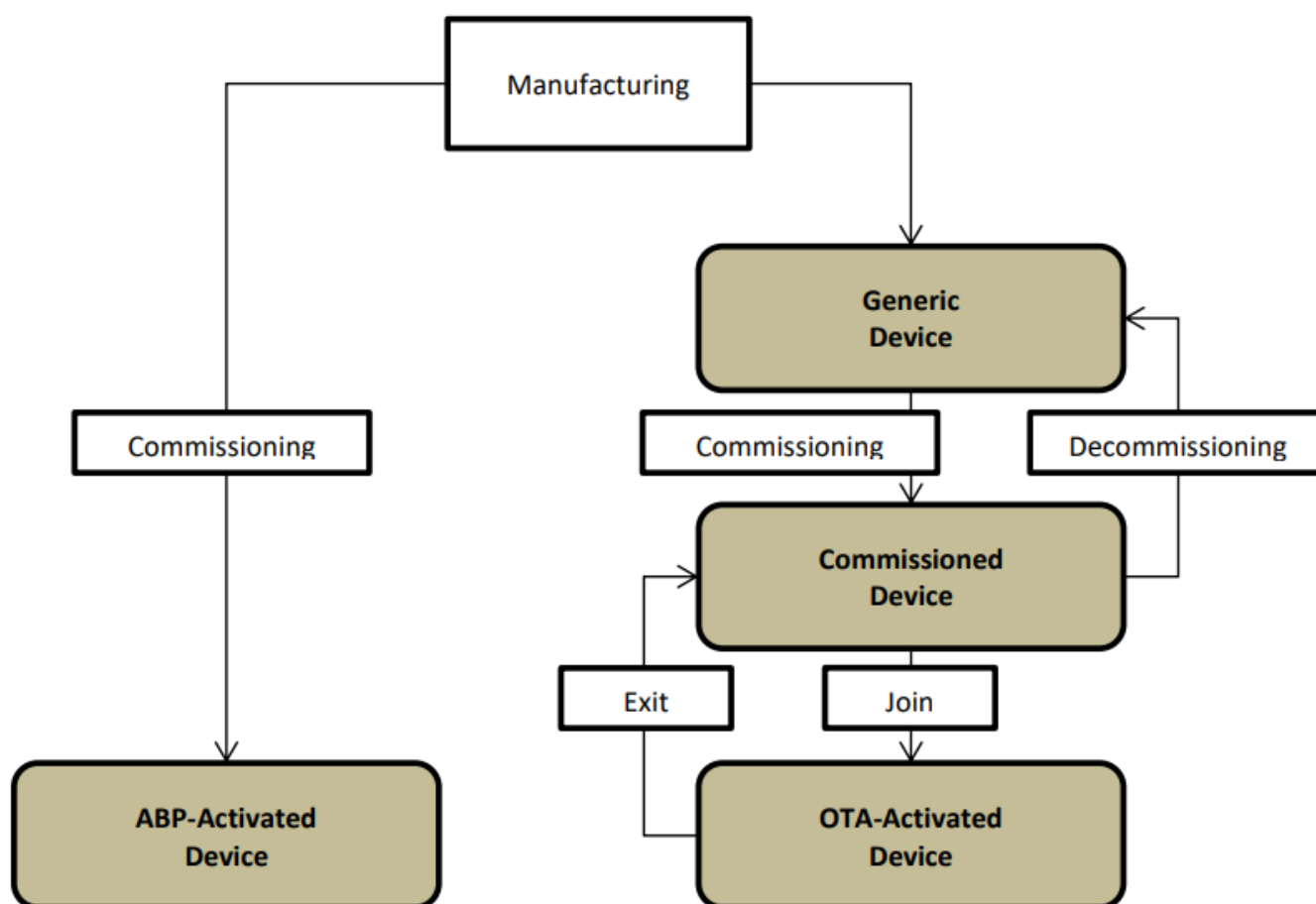


Рисунок 2.8 — Модель частной сети LoRaWAN

Классы устройств:

- **Класс А** (асинхронный) — устройство открывает два коротких окна приёма (RX1, RX2) только после каждой передачи uplink. Минимальное потребление.
- **Класс В** (синхронный) — дополнительно открывает окна приёма по расписанию, синхронизированному с сетью.
- **Класс С** (непрерывный приём) — постоянно слушает канал (кроме времени передачи). Максимальная доступность для downlink.

2.2.2. Активация конечного устройства

Существуют два вида активации конечного устройства в сети LoRaWAN:

1. **ABP (Activation by Personalization)** — статическая активация.

В данном случае все ключи и параметры устройства задаются вручную до подключения к сети, без использования Join-request. Конечное устройство получает DevAddr, AppSKey, сетевые ключи сессии (SNwkSIntKey, FNwkSIntKey, NwkSEncKey). После включения устройство сразу может передавать данные, не проходя активацию.

2. OTAA (Over-The-Air Activation) — динамическая активация.

Процедура включает Join-request и получение сгенерированных ключей от сервера активации (Join Server):

- Изначально конечное устройство — Generic Device (содержит только DevEUI и корневые ключи AppKey, NwkKey).
- После привязки к сети (commissioning) оно становится Commissioned Device.
- При успешной активации — OTA Activated Device.
- Конечное устройство может быть деактивировано через процедуру Exit; повторная активация приведёт к созданию нового LoRaWAN сеанса.

3 bits	21-N bits	N bits
Type	RFU	ID

Рисунок 2.9 — Диаграмма состояний устройства при двух типах активации

2.2.3. Форматы NetID и DevAddr

NetID — 24-битный идентификатор сети. Значения 0 и 1 зарезервированы для экспериментальных или не использующих роуминг сетей.

Структура NetID:

- 3 старших бита задают тип NetID.
- 6, 9 или 21 (в зависимости от типа) младших битов занимает ID, выданный LoRa Alliance.
- Оставшиеся биты помечаются как RFU и записаны в ноль.

DevAddr — 32-битный идентификатор конечного устройства. Структура: Type Prefix (L бит) + NwkID (M бит) + NwkAddr (N бит). Размеры L, M, N зависят от типа NetID.

Для NetID Type 0 (наиболее распространённый для частных сетей):

- Type Prefix: 1 бит (значение 0)
- NwkID: 6 бит
- NwkAddr: 25 бит

Это обеспечивает до 33,5 миллиона адресов устройств в одной частной сети.

2.2.4. Пакеты LoRaWAN

Структура PHY payload: MHDR (1 байт) + MACPayload или Join-Request + MIC (4 байта).

MACPayload состоит из: FHDR (Frame Header) + FPort (опционально) + FRMPayload (опционально).

FHDR включает: DevAddr (4 байта) + FCtrl (1 байт) + FCnt (2 байта) + FOpts (0–15 байт MAC-команд).

Ключевые поля пакета uplink:

Поле	Описание
DevAddr	Локальный адрес устройства в сети
FCntUp	Счётчик uplink сообщений
ADRbit	Флаг использования Adaptive Data Rate
DataRate	Индекс режима передачи
RSSI	Индикатор силы сигнала
SNR	Отношение сигнал/шум

MAC-команды (передаются в FOpts или как отдельный FRMPayload при FPort=0):

- LinkCheckReq/Ans — запрос проверки связи
- LinkADRRReq/Ans — запрос изменения скорости передачи
- DutyCycleReq/Ans — управление коэффициентом занятости канала
- RXParamSetupReq/Ans — настройка параметров окон приёма

2.2.5. Adaptive Data Rate (ADR)

ADR — механизм оптимизации использования радиоканала, при котором сетевой сервер управляет параметрами передачи конечного устройства (SF, BW, TX Power) для минимизации энергопотребления и времени занятости канала при сохранении надёжности связи.

Алгоритм ADR на стороне сервера:

1. Сервер накапливает историю SNR последних 20 пакетов uplink.
2. Вычисляется максимальное SNR: SNR_max.
3. Запас по линку: Margin = SNR_max – SNR_required(DR_current) – 10 дБ.
4. Если Margin > 0, сервер снижает SF (повышает DR) на Margin / 3 ступени.
5. Если устройство не отвечает на несколько последовательных запросов LinkADRRReq, оно увеличивает SF (снижает DR) вплоть до максимального.

Компромисс: чем выше DR (ниже SF), тем короче время занятости канала (time-on-air) и ниже энергопотребление, но меньше дальность и помехоустойчивость.

2.3. Meshtastic

Meshtastic — открытый проект, реализующий децентрализованную mesh-сеть для LoRa-приёмопередатчиков. В отличие от LoRaWAN, не требует централизованной инфраструктуры (шлюзов, серверов).

2.3.1. Модель сети

Meshtastic использует децентрализованную mesh-топологию на основе управляемого затопления (managed flooding). Каждый узел, получивший пакет впервые, ретранслирует его (с учётом счётчика `hop_limit`). Отсутствует единый координатор — сеть самоорганизующаяся.

Роли узлов:

- **CLIENT** — стандартный узел с интерфейсом пользователя (Bluetooth/WiFi/Serial).
- **CLIENT_MUTE** — принимает пакеты, но не ретранслирует.
- **ROUTER** — оптимизирован для ретрансляции, не имеет пользовательского интерфейса.
- **ROUTER_CLIENT** — совмещает функции ROUTER и CLIENT.
- **REPEATER** — ретранслирует пакеты с максимальным приоритетом.
- **TRACKER** — оптимизирован для периодической отправки данных о местоположении.

Параметр **hop_limit** (по умолчанию 3) ограничивает максимальное число ретрансляций пакета и предотвращает заикливание. Уникальный идентификатор пакета (`packet_id + sender`) позволяет каждому узлу отбросить уже обработанный пакет.

2.3.2. Стек протоколов

Meshtastic реализует полный стек уровней:

Уровень	Реализация
Физический	LoRa PHY (CSS, SF, BW, CR, LDRO)
Канальный	MeshPacket: dest (4 байта), sender (4 байта), packet_id (4 байта), flags (1 байт), channel_hash (1 байт)
Сетевой	Управляемое затопление, hop_limit, дедупликация пакетов
Транспортный	Надёжная доставка (ACK), сессии для длинных пакетов
Прикладной	Protobuf: Data, Position, Telemetry, NodeInfo, Routing и др.

Структура MeshPacket на канальном уровне:

```

dest:      4 байта — адрес назначения (0xFFFFFFFF = broadcast)
sender:    4 байта — адрес отправителя
packet_id: 4 байта — уникальный идентификатор пакета
flags:     1 байт  — hop_limit (3 бита), want_ack, via_mqtt и др.
channel_hash: 1 байт — XOR имени канала и ключа шифрования
reserved:  2 байта
data:      N байт  — зашифрованные данные (AES-128/256 CTR)

```

2.3.3. Пресеты модема и частотные планы

Meshtastic определяет набор предустановленных конфигураций модема:

Пресет	BW (кГц)	SF	CR	LDRO
--------	----------	----	----	------

Пресет	BW (кГц)	SF	CR	LDRO
SHORT_TURBO	500	7	4/5	—
SHORT_FAST	250	7	4/5	—
SHORT_SLOW	250	8	4/5	—
MEDIUM_FAST	250	9	4/5	—
MEDIUM_SLOW	250	10	4/5	—
LONG_FAST	250	11	4/5	—
LONG_TURBO	500	11	4/5	—
LONG_MODERATE	125	11	4/8	Да

Частотные планы по регионам (слоты для LONG_FAST / BW=250 кГц):

Регион	Слот 0 (МГц)	Слот 1 (МГц)	Слот N
EU868	869,525	—	1 слот
RU	868,825	869,075	2 слота
EU433	433,125	433,375	4 слота

Принцип слотирования позволяет нескольким каналам работать независимо в пределах одного частотного плана. Узлы с разными слотами не интерферируют друг с другом.

2.3.4. Безопасность

Meshtastic шифрует полезную нагрузку пакетов с помощью AES в режиме CTR (Counter). Каждый канал имеет собственный ключ.

Вычисление хеша канала:

```
channel_hash = XOR(все байты имени канала) XOR (все байты ключа)
```

Формирование вектора инициализации шифра:

```
IV = packet_id (8 байт, little-endian) || sender (8 байт, little-endian)
```

Поддерживаются ключи AES-128 (16 байт) и AES-256 (32 байта). Специальное значение ключа **AQ==** (base64 от 0x01) соответствует стандартному публичному ключу по умолчанию, используемому для широковещательного канала.

2.3.5. Интерфейсы управления

Прошивка Meshtastic поддерживает несколько интерфейсов управления:

- **Bluetooth** — для мобильного приложения Meshtastic.
- **Serial** — для подключения через UART/USB.
- **TCP/IP** — для управления через сеть (используется в данной работе).

Конфигурация через TCP осуществляется с помощью Meshtastic Python API. Структура конфигурации описана в Protobuf-схеме (`config_pb2`): `Config.LoRaConfig`, `Config.DeviceConfig`, `Config.NetworkConfig` и др.

2.4. Reticulum

Reticulum — криптографически адресованный сетевой стек для построения децентрализованных сетей. Разработан как альтернатива традиционным IP-сетям для среды с ненадёжными каналами связи. Поддерживает работу поверх LoRa через интерфейс RNode.

2.4.1. Модель сети

Reticulum не использует централизованную инфраструктуру и глобальные таблицы маршрутизации. Основные типы узлов:

- **Interface** — физический или виртуальный интерфейс для подключения к среде передачи (LoRa, TCP, Serial и т.д.).
- **Transport Node** — узел, который накапливает маршрутную информацию и пересылает пакеты. Аналог маршрутизатора.
- **Destination** — конечная точка адресации. Адрес вычисляется как усечённый хеш публичного ключа Ed25519 (16 байт), что обеспечивает криптографическую самосертификацию адресов.

Отсутствие фиксированной топологии: узлы обнаруживают маршруты динамически через механизм Announce.

2.4.2. Адресация и идентификаторы

Типы Destination:

- **SINGLE** — один получатель, шифрование end-to-end.
- **GROUP** — групповая адресация.
- **PLAIN** — без шифрования.
- **LINK** — установленный двусторонний канал с совершенной прямой секретностью (PFS).

Механизм **Announce**: узел периодически рассылает объявление со своим адресом и публичным ключом. Transport Node'ы накапливают эти объявления и строят таблицу маршрутизации.

2.4.3. Маршрутизация и транспорт

Pathfinding: Transport Node накапливает Announce-пакеты, отслеживает количество hop'ов до каждого Destination и выбирает оптимальный маршрут.

Типы пакетов:

- **Announce** — объявление о существовании Destination.
- **Packet** — передача данных без предварительной установки соединения.
- **Link Request / Proof** — установка зашифрованного канала с PFS.

Поддержка LoRa через **RNode** — специализированная прошивка для LoRa-модулей (LILYGO T-Beam и др.), реализующая интерфейс KISS over Serial для Reticulum.

2.4.4. Сравнение Meshtastic и Reticulum

Критерий	Meshtastic	Reticulum
Модель безопасности	Канальное шифрование (AES, общий ключ канала)	End-to-end (Ed25519, X25519, AES-256 GCM)
Маршрутизация	Управляемое затопление (flooding)	Реактивная маршрутизация по Announce
Адресация	32-битный MAC-адрес (NodeID)	Хеш публичного ключа (16 байт)
Инфраструктура	Не требуется	Не требуется
Масштабируемость	Ограничена hop_limit и duty cycle	Лучше за счёт адресной маршрутизации
Прошивка для LoRa	Meshtastic firmware	RNode firmware
Зрелость экосистемы	Высокая (приложения, веб-интерфейс)	Средняя

2.5. Обзор существующих решений эмуляции и симуляции LoRa-сетей

Анализ существующих инструментов позволяет выявить их ключевые ограничения применительно к задаче эмуляции полного стека Meshtastic.

GNURadio + gr-lora_sdr [1] — реализует точную DSP-цепочку физического уровня LoRa. Достоинство: полная верификация физического уровня. Ограничение: нет поддержки сетевого уровня и реальной прошивки; требует значительных вычислительных ресурсов для реального времени.

ns-3 (LoRaWAN module) — дискретно-событийный симулятор с моделью LoRaWAN. Достоинство: масштабируемость, поддержка большого числа узлов. Ограничение: упрощённая вероятностная модель физического уровня; нет поддержки Meshtastic; не использует реальную прошивку.

OMNeT++ / FLoRa — аналогичен ns-3 по классу инструментов. Предоставляет вероятностные модели LoRa-канала и LoRaWAN. Ограничение: те же, что у ns-3.

LoRaSim — упрощённый Python-симулятор, оценивающий вероятность успешной доставки пакета (PDR) в сети LoRaWAN. Ограничение: отсутствует какая-либо реализация сетевого стека.

Meshtasticator — симулятор, специально разработанный для Meshtastic. Реализует вероятностную модель канала и алгоритм маршрутизации Meshtastic. Достоинство: учитывает специфику протокола

Meshtastic. Ограничение: использует упрощённую вероятностную модель физического уровня (без DSP-цепочки); не запускает реальную прошивку.

Решение	Физический уровень	Сетевой уровень	Реальная прошивка	GUI
GNURadio + gr-lora_sdr	Точный DSP	Нет	Нет	Нет
ns-3 (LoRaWAN)	Вероятностная	LoRaWAN	Нет	Нет
OMNeT++ / FLoRa	Вероятностная	LoRaWAN	Нет	Нет
LoRaSim	PDR-модель	Нет	Нет	Нет
Meshtasticator	Вероятностная	Meshtastic	Нет	Нет
LoRaEmulator (данная работа)	DSP (CSS, AWGN)	Meshtastic	Да	Да

Таблица 2.1 — Сравнение существующих решений

Вывод: ни одно из рассмотренных решений не совмещает точную DSP-модель физического уровня LoRa с запуском реальной прошивки Meshtastic и визуальным управлением топологией сети. Разработанный LoRaEmulator заполняет эту нишу, реализуя концепцию SITL для полного стека Meshtastic.

3. Программное обеспечение

3.1. Архитектура системы

Разработанный эмулятор реализует концепцию Software-in-the-Loop (SITL). Выбор SITL обусловлен возможностью запуска встроенного программного обеспечения узлов в виртуальной среде на хост-компьютере. SITL позволяет эмулировать все уровни программно, снижает затраты и обеспечивает высокую воспроизводимость экспериментов — в отличие от HITL, где используются реальные физические компоненты.

Пользовательские настройки приёмопередатчиков и отправляемых сообщений передаются в запущенный процесс Meshtastic через TCP-сокеты в Meshtastic Python API. После этого переданные параметры или сообщения передаются на виртуальные физические узлы через реализованный виртуальный драйвер (KISS over UDP). Виртуальные физические узлы взаимодействуют между собой через эмулятор канала связи, принятые сообщения передаются обратно на уровень выше до пользовательского уровня приложений.

Система имеет четырёхуровневую структуру: GUI ↔ Python-оркестратор ↔ LoRaSDR (Rust/FutureSDR) ↔ Docker-контейнеры с прошивкой Meshtastic.

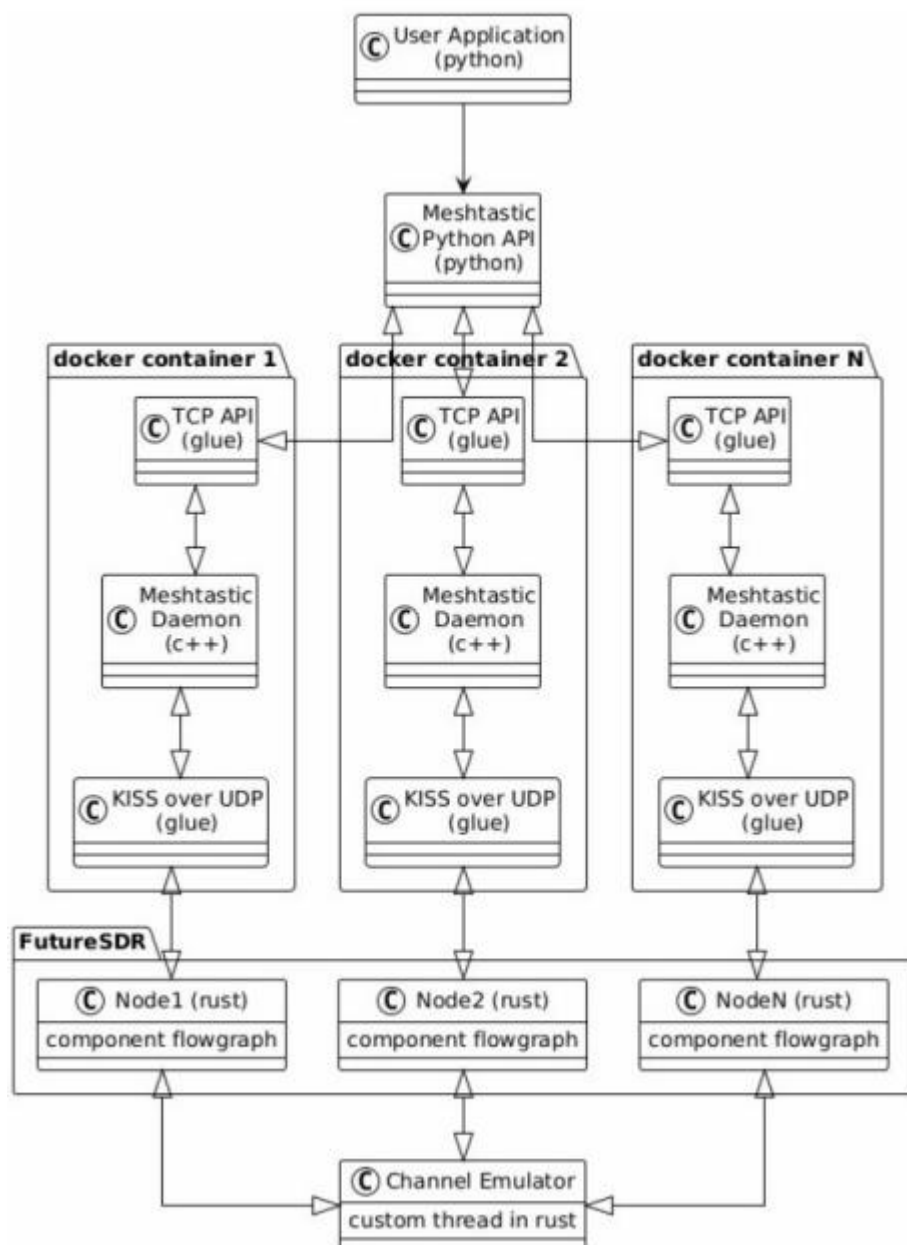


Рисунок 3.1 — Многоуровневая архитектура SITL-эмулятора

Архитектура состоит из следующих компонентов:

- **User Application** — уровень пользовательского приложения (Python GUI).
- **Meshtastic Python API** — клиент для TCP-сервера в процессе Meshtastic. Используется для программной конфигурации узлов.
- **TCP API (glue)** — сервер, реализующий управление процессом Meshtastic внутри Docker-контейнера.
- **Meshtastic Daemon** — процесс `meshtasticd`, та же самая программа, что работает на реальных устройствах (платформа `native_virtual`).
- **KISS over UDP (glue)** — компонент, реализующий управление узлом и приём/передачу данных с помощью KISS-команд через UDP-сокеты.
- **Node 1..N (FutureSDR)** — узлы приёмопередатчиков, описываемые с помощью графов потоков обработки сигналов в FutureSDR.
- **Channel Emulator** — эмулятор канала связи, связывающий все узлы между собой.

Обоснование выбора технологий:

FutureSDR vs GNU Radio: GNU Radio ориентирован на визуальное проектирование потоков обработки сигналов в среде GNU Radio Companion (GRC). Данный подход накладывает ограничение на гибкость и затрудняет реализацию нестандартных сценариев и интеграцию. FutureSDR использует противоположный подход — граф описывается программно в виде набора объектов-блоков и их соединений. Это даёт свободу действий при создании потоков обработки и обеспечивает высокую производительность.

Crossbeam: связь между моделью радиоканала и блоками обработки сигналов FutureSDR реализована с помощью высокопроизводительных каналов библиотеки crossbeam. Crossbeam канал позволяет нескольким потокам одновременно отправлять и принимать данные. В отличие от разделяемой памяти (shared memory), crossbeam использует подход передачи сообщений, что упрощает синхронизацию и обеспечивает естественную интеграцию с асинхронной архитектурой FutureSDR.

3.2. Реализация физического уровня (LoRaSDR)

3.2.1. Модель канала связи

Модель канала связи выполняется в отдельном потоке (класс `ChannelProcessor`). Реализует распределение IQ-сигналов между всеми виртуальными узлами.

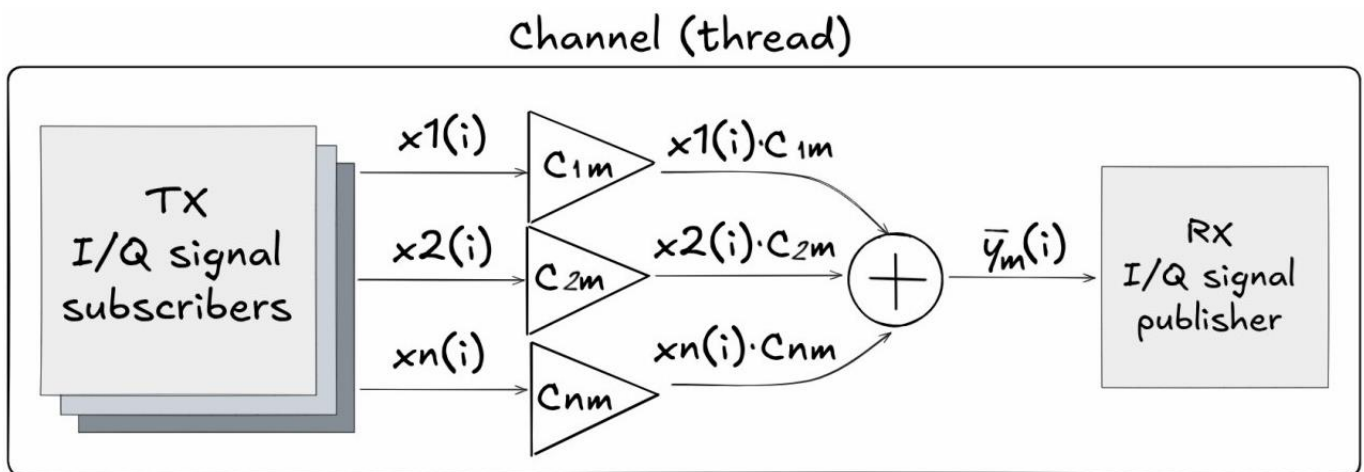


Рисунок 3.2 — Модель работы канала связи

По каналам crossbeam передаются объекты типа `IqFrame`:

```

pub struct IqFrame {
    pub start_index: u64,           // временная метка (номер фрейма)
    pub samples: [Complex32; 1024], // буфер комплексных отсчётов
    pub fc: f64,                   // центральная частота передатчика
    pub bw: f32,                   // ширина полосы
}
  
```

где `start_index` — временная метка для синхронизации фреймов между узлами, `samples` — буфер комплексных IQ-отсчётов размером 1024 сэмпла.

Алгоритм работы `ChannelProcessor::run()`:

1. На каждой итерации проверяются очереди `tx_out` всех узлов (неблокирующий `try_recv`).
2. Полученные фреймы накапливаются в `pending_frames`, индексированных по `start_index`.
3. Для каждого получателя `m` и каждого переданного фрейма от узла `n`:
 - Вычисляется расстояние между узлами: $d = \sqrt{(x_n - x_m)^2 + (y_n - y_m)^2}$.
 - Применяется коэффициент затухания: $C_{nm} = (d + 1)^{-3} / \sqrt{2}$.
 - Сигнал масштабируется и добавляется в суммарный буфер: `txbuf[i] += frame.samples[i] * C_nm`.
4. Суммарный сигнал отправляется в очередь `rx_in` узла-получателя.

Модель затухания (d^{-3}) соответствует распространению сигнала в условиях, промежуточных между свободным пространством (d^{-2}) и городской застройкой (d^{-4}). В случае $d = 0$ (узел сам себе) пакет не отправляется.

3.2.2. Модель узла в FutureSDR

Каждый виртуальный узел представляет собой два независимых графа FutureSDR: **путь передачи** (TX) и **путь приёма** (RX).

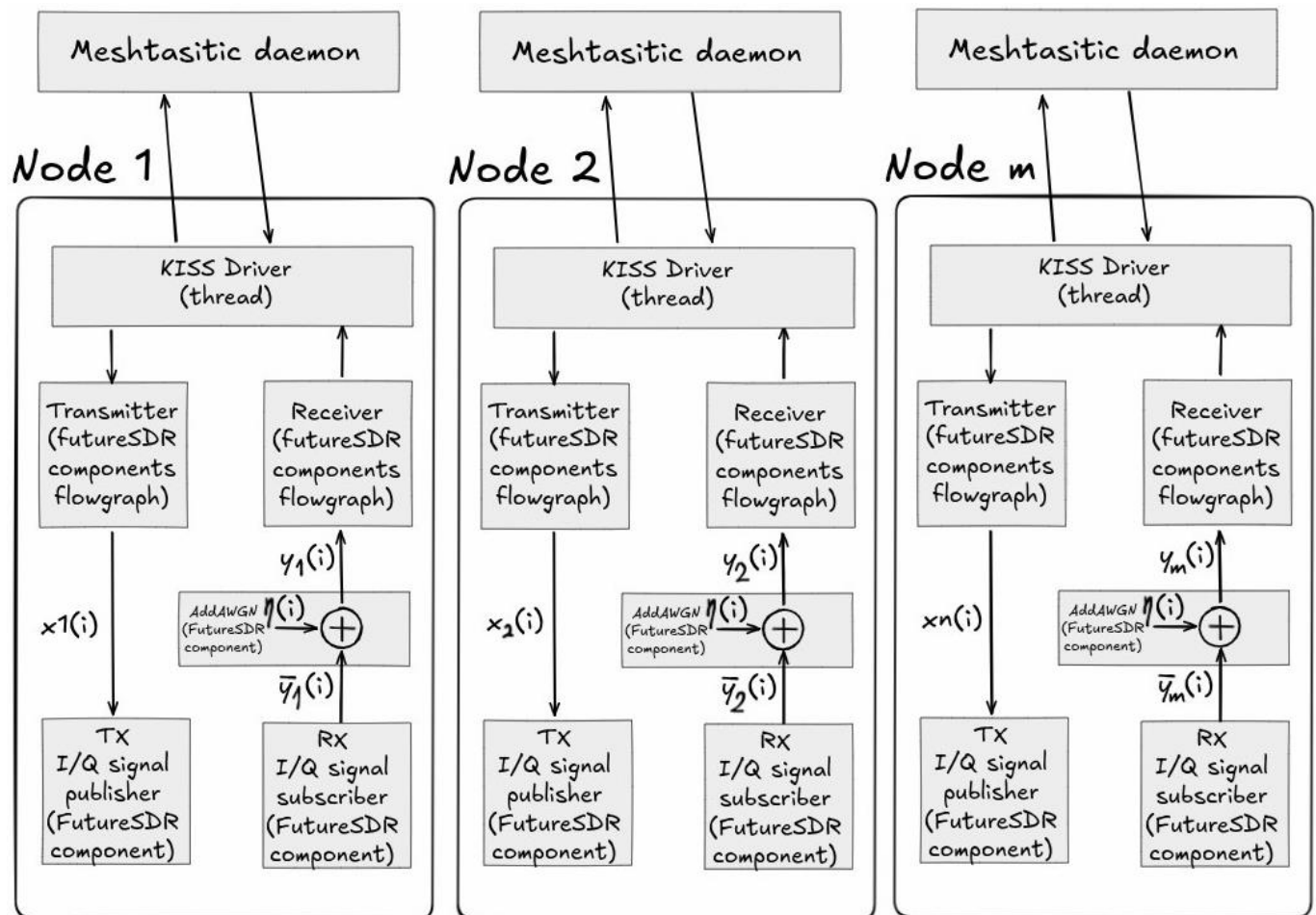


Рисунок 3.3 — Взаимодействие потоков эмуляции с узлами в FutureSDR

Путь передачи отвечает за преобразование данных, поступающих от уровня сети (KISS from Meshtastic daemon), в IQ-сигнал:

KISS (UDP) → Transmitter → ChannelPublisher → tx_out (crossbeam)

- **Transmitter** — реализует полную цепочку кодирования LoRa: кодирование Грея → интерливинг → код Хэмминга → CSS-модуляция (chirp). Параметризуется SF, BW, CR, LDRO.
- **ChannelPublisher** — блок FutureSDR, который читает IQ-буфер и публикует **IqFrame** в канал crossbeam.

Путь приёма обрабатывает суммарный IQ-сигнал от канала и извлекает байты:

```
rx_in (crossbeam) → ChannelSubscriber → AddAWGN → FrameSync →
→ FftDemod → GrayMapping → Deinterleaver → HammingDecoder →
→ HeaderDecoder → Decoder → BlobToUdp (KISS to Meshtastic)
```

- **ChannelSubscriber** — блок FutureSDR, читающий **IqFrame** из crossbeam-канала и подающий IQ-отсчёты в граф.
- **AddAWGN** — добавляет аддитивный белый гауссовский шум с параметром σ (задаётся при создании узла).
- **FrameSync** — обнаруживает преамбулу, определяет sync word, выравнивает поток символов.
- **FftDemod** — FFT-демодуляция: вычисляет аргумент максимума FFT для каждого символа.
- **GrayMapping** — обратное преобразование Грея: $s \rightarrow$ двоичный код.
- **Deinterleaver** — обратное диагональное перемежение.
- **HammingDecoder** — декодирование кода Хэмминга, исправление ошибок.
- **HeaderDecoder** — разбор заголовка explicit-кадра, извлечение CR, наличия CRC, длины payload.
- **Decoder** — извлечение полезной нагрузки, проверка CRC.
- **BlobToUdp** — отправка декодированных байт на порт **remote_port** Meshtastic daemon по UDP (KISS-формат).

Для каждого пресета модема (ShortTurbo, ShortFast, ..., LongModerate) создаётся отдельный полный путь приёма, что обеспечивает возможность смены пресета без перезапуска.

3.2.3. UDP-сервер узла (интерфейс к Meshtastic daemon)

Каждый узел запускает асинхронный UDP-сервер на порту **local_port**, который принимает KISS-пакеты от Meshtastic daemon и передаёт их в FutureSDR Transmitter:

```
async fn server_task_body(handle: FlowgraphHandle, txs_ids: Vec<BlockId>,
local_port: u16) {
    let socket = UdpSocket::bind(format!("127.0.0.1:{}",
local_port)).await;
    loop {
        let (n, peer) = socket.recv_from(&mut buf).await;
        handle.call(txs_ids[selected_tx], "msg", Pmt::Blob(payload)).await;
        socket.send_to(&[0xC0, 0x0F, 0x00, 0xC0], peer).await; // KISS ACK
    }
}
```

Ответ `[0xC0, 0x0F, 0x00, 0xC0]` — стандартный KISS-отклик, подтверждающий приём пакета прошивкой.

3.3. Сетевой уровень: адаптация Meshtastic

3.3.1. Платформа `native_virtual`

Прошивка Meshtastic собирается для платформы `native_virtual` с помощью PlatformIO. В отличие от аппаратных платформ (ESP32, nRF52), `native_virtual` компилируется под Linux x86_64 и запускается как обычный процесс. Аппаратные драйверы (SPI, GPIO) заменяются программными заглушками.

Команда сборки:

```
cd meshtastic_firmware && pio run -e native_virtual
```

Результирующий бинарный файл: `.pio/build/native_virtual/meshtasticd`

3.3.2. KISS-протокол как интерфейс SDR ↔ прошивка

KISS (Keep It Simple, Stupid) — протокол инкапсуляции пакетов, изначально разработанный для TNC (Terminal Node Controller) в системах пакетной радиосвязи. В данной работе используется для организации интерфейса между Meshtastic daemon и LoRaSDR.

Структура KISS-фрейма:

```
FEND (0xC0) | CMD (1 байт) | DATA (N байт) | FEND (0xC0)
```

Специальные символы в потоке данных экранируются: `0xC0` → `0xDB 0xDC`; `0xDB` → `0xDB 0xDD`.

В `KissDriver.h` (адаптированная прошивка Meshtastic) реализованы:

- Отправка пакета: упаковка LoRa-пакета в KISS-фрейм и отправка по UDP на `local_port` LoRaSDR.
- Приём пакета: приём UDP-датаграммы от LoRaSDR с `remote_port`, распаковка KISS-фрейма, передача в Meshtastic stack.

Распределение портов:

- `local_port` — UDP-порт, на котором LoRaSDR принимает пакеты от Meshtastic (Meshtastic → LoRaSDR).
- `remote_port` — UDP-порт Meshtastic, на который LoRaSDR отправляет декодированные пакеты (LoRaSDR → Meshtastic).
- `tcp_port` — TCP-порт для управления узлом через Meshtastic Python API.
- `web_port` — HTTP-порт встроенного веб-интерфейса Meshtastic.

3.3.3. Программная конфигурация узлов через TCP

После запуска контейнера Python-оркестратор подключается к прошивке через TCP и устанавливает параметры LoRa:

```
def onConnection(self, interface, **kwargs):
    node = self.interface.localNode
    lc = node.localConfig
    lc.lora.region = config_pb2.Config.LoRaConfig.RegionCode.RU
    lc.lora.use_preset = True
    lc.lora.modem_preset =
config_pb2.Config.LoRaConfig.ModemPreset.LONG_FAST
    node.writeConfig("lora")
```

Это позволяет динамически изменять конфигурацию узла (регион, пресет модема) без пересборки прошивки.

3.4. Изоляция узлов на базе Docker

Каждый виртуальный узел запускается в отдельном Docker-контейнере. Это обеспечивает:

- Изоляцию файловой системы и процессов между узлами.
- Единое окружение для всех узлов (PlatformIO, зависимости).
- Автоматическую очистку ресурсов при остановке (`auto_remove=True`).

Жизненный цикл контейнера:

1. При первом запуске проверяется наличие образа `meshtastic-firmware-dev`. Если образ отсутствует — собирается из `docker/Dockerfile`.
2. Проверяется наличие скомпилированного бинарника `meshtasticd`. Если отсутствует — выполняется сборка в Docker (`pio run -e native_virtual`).
3. Запускается контейнер с командой:

```
./meshtasticd -s -v -p {tcp_port} -l {local_port} -r {remote_port} -w
{web_port} -h {MAC_address}
```

4. После запуска открывается окно xterm с логами контейнера для диагностики.
5. Через 8 секунд (время инициализации прошивки) устанавливается TCP-соединение через Meshtastic Python API для конфигурации.
6. При остановке контейнер останавливается и удаляется (`container.stop()` + `container.remove(force=True)`).

Монтирование рабочего каталога позволяет всем контейнерам использовать один и тот же бинарник прошивки и веб-интерфейс:

```
volumes={workspace_path: {'bind': '/home/user/workspace/', 'mode': 'rw'}}
```

3.5. Графический интерфейс

Графический интерфейс реализован с использованием фреймворка PyQt6 по архитектурному паттерну MVC.

Компоненты MVC:

- *Model*: **Node** (конфигурация и состояние виртуального узла), **Project** (коллекция узлов).
- *View*: **MapScene** (сцена QGraphicsScene с узлами), **NodeItem** (графическое представление узла), **GraphicsView** (виджет с масштабированием и прокруткой), **dialogs** (диалоги конфигурации).
- *Controller*: **NodeController** (управление жизненным циклом узлов), **ProjectController** (сохранение/загрузка проектов), **EmulationController** (групповые операции).

Визуализация топологии:

Узлы размещаются на бесконечном 2D-холсте. Положение узла в системе координат холста используется как входной параметр для модели затухания в ChannelProcessor. Каждый узел отображается как цветной круг с подписью (последние 4 символа MAC-адреса + состояние):

Состояние	Цвет	Описание
CREATED	Серый	Узел создан, не запущен
STARTING	Жёлтый	Контейнер запускается
RUNNING	Зелёный	Узел активен
STOPPING	Жёлтый	Контейнер останавливается
STOPPED	Синий	Узел остановлен пользователем
ERROR	Красный	Ошибка запуска или сбой

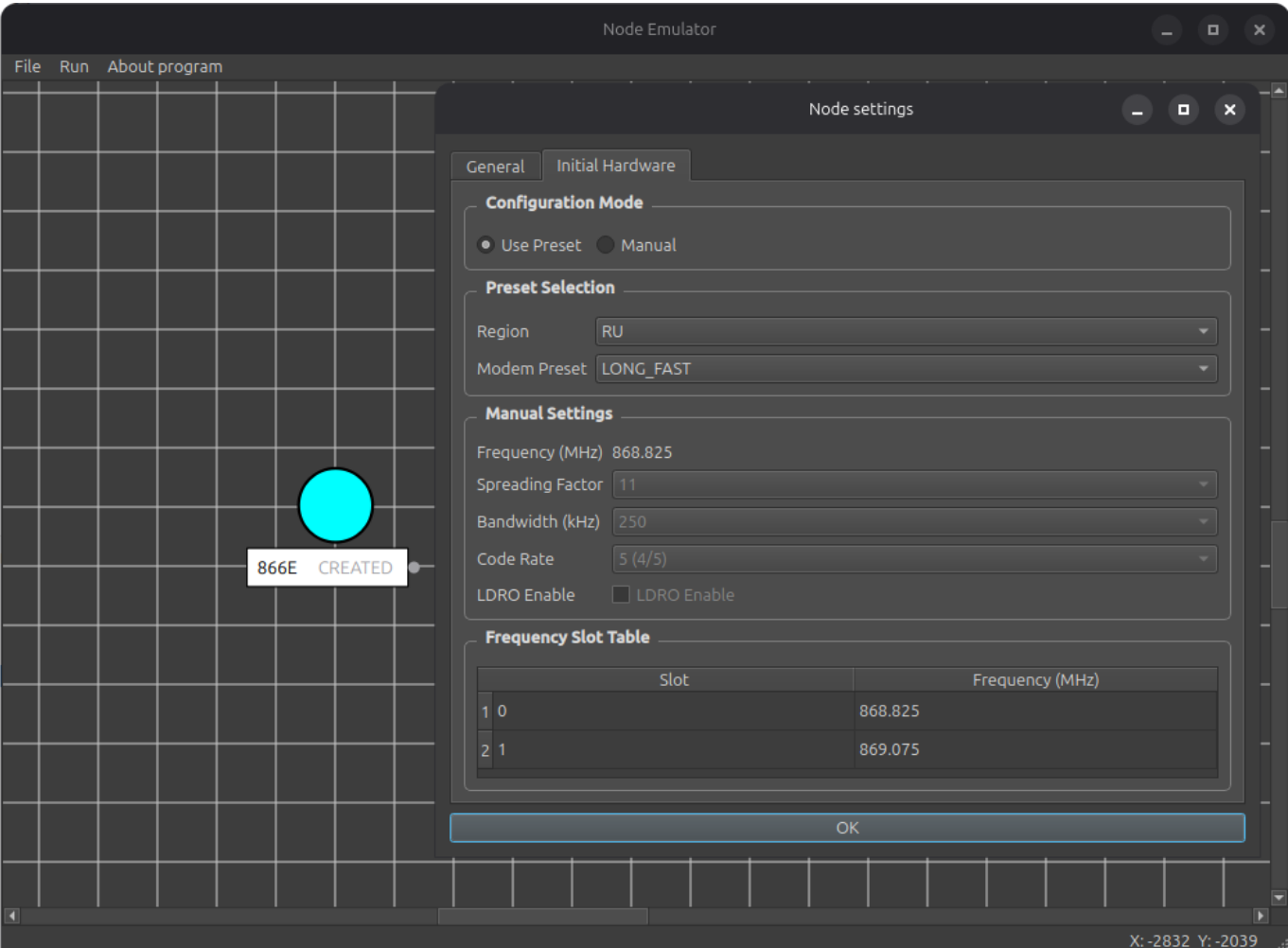


Рисунок 3.4 — Графический интерфейс для конфигурации эмуляции сети

Диалог конфигурации узла (вкладка «Initial Hardware»):

- Режим конфигурации: «Use Preset» (выбор региона и пресета модема) или «Manual» (ручная установка SF, BW, CR, LDRO).
- Таблица доступных частотных слотов (автоматически заполняется при выборе пресета и региона).
- Сетевые порты: local_port, remote_port, web_port, tcp_port.

Результат работы системы:

После запуска двух и более виртуальных узлов они видят друг друга в сети Meshtastic и могут обмениваться сообщениями через виртуальный радиоканал. Принятые сообщения отображаются во встроенном веб-интерфейсе Meshtastic каждого узла.

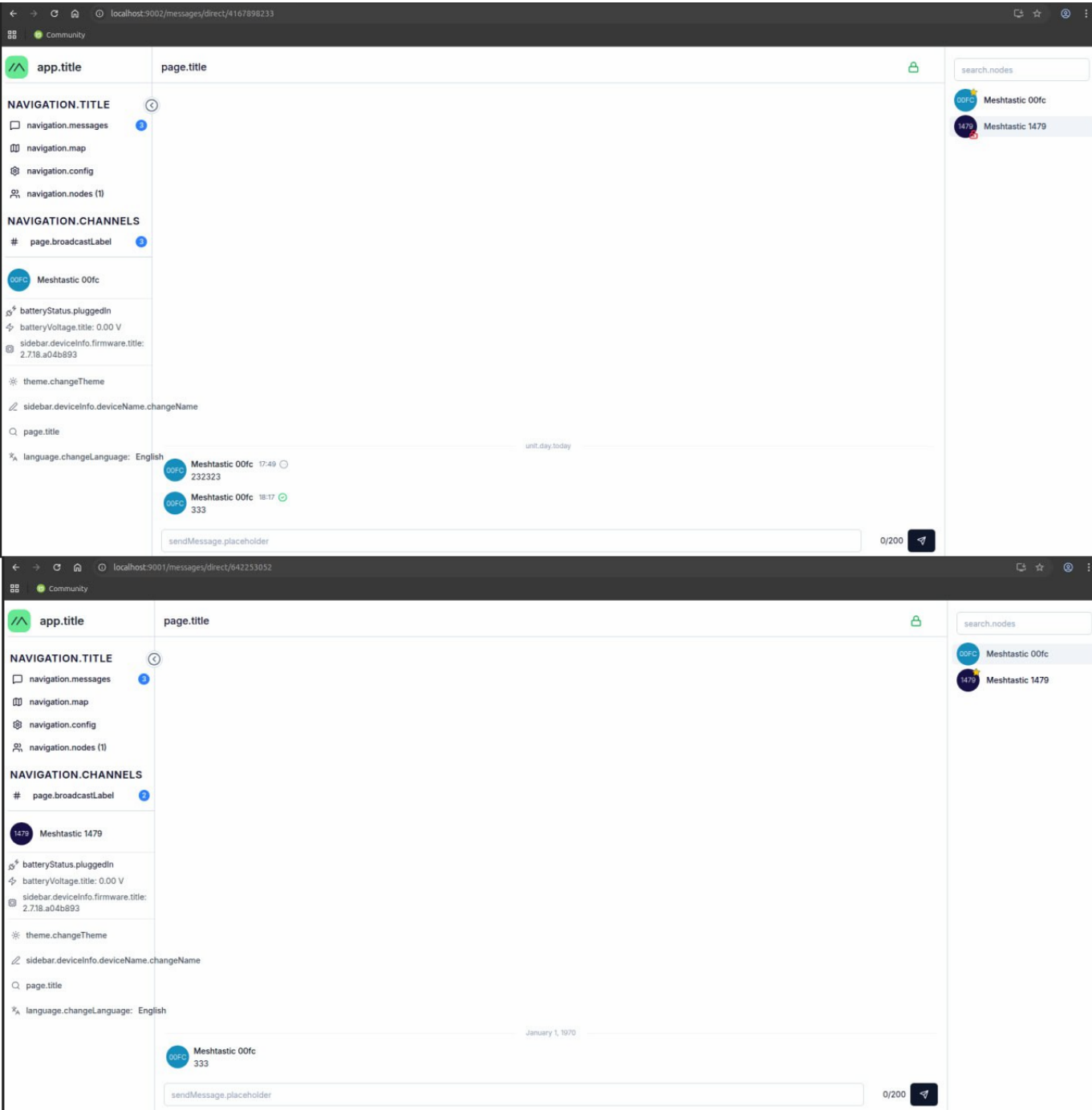


Рисунок 3.5 — Передача текстового сообщения между двумя виртуальными узлами

На рисунке 3.5 показан веб-интерфейс двух виртуальных узлов: узел «Meshtastic 00fc» отправил сообщение «222321», которое корректно отображается у узла «Meshtastic 1470», подтверждая работоспособность полного стека эмуляции.

4. Заключение

В настоящей работе разработан прототип SITL-эмулятора LoRaEmulator, подтверждающий возможность эмуляции полного стека Meshtastic без использования физического оборудования.

В ходе работы были решены следующие задачи:

1. **Разработана модель канала связи** — реализован класс `ChannelProcessor`, обеспечивающий маршрутизацию IQ-кадров между виртуальными узлами с учётом затухания (d^{-3}) и AWGN-шума.

2. **Реализован физический уровень LoRa** — в рамках LoRaSDR разработана полная DSP-цепочка модуляции и демодуляции CSS-сигнала (Transmitter, FrameSync, FftDemod, GrayMapping, Deinterleaver, HammingDecoder, HeaderDecoder, Decoder) на базе фреймворка FutureSDR.
3. **Разработан механизм связи узлов с моделью канала** — через блоки `ChannelPublisher` и `ChannelSubscriber` на базе высокопроизводительных каналов библиотеки crossbeam, обеспечивающих потокобезопасный обмен IQ-фреймами.
4. **Реализован драйвер эмуляции физического интерфейса Meshtastic** — виртуальный `KissDriver`, заменяющий аппаратный SPI/UART интерфейс на KISS over UDP, обеспечивает взаимодействие прошивки Meshtastic с LoRaSDR.
5. **Адаптирована прошивка Meshtastic** для работы в виртуальной среде (платформа `native_virtual`); каждый экземпляр прошивки изолирован в Docker-контейнере с параллельным TCP-интерфейсом для программной конфигурации.
6. **Разработан графический интерфейс** по паттерну MVC, обеспечивающий визуальное размещение узлов на 2D-топологии, конфигурацию LoRa-параметров, управление жизненным циклом контейнеров и сохранение/загрузку конфигурации сети.

Разработанное решение обеспечивает эмуляцию взаимодействия виртуальных узлов через виртуальный радиоканал, включая передачу пакетов и их отображение во встроенном веб-интерфейсе Meshtastic. Научная новизна заключается в создании SITL-инструмента, ориентированного на полную эмуляцию всех уровней стека, включая виртуальный физический уровень и виртуальный радиоканал.

Направления дальнейшего развития:

- Реализация частотной избирательности канала (поддержка одновременной работы узлов с разными пресетами и слотами).
- Добавление мобильности узлов (динамическое изменение позиции в ходе эмуляции).
- Реализация многолучевой модели канала (Rayleigh/Rician fading).
- Экспорт сетевых метрик (PDR, задержка, hop-count) для анализа производительности.
- Поддержка протокола Reticulum поверх виртуального LoRa-канала.

Список использованных источников

1. Tapparel J., Burg A. Design and Implementation of LoRa Physical Layer in GNU Radio. — EPFL, 2024.
2. Yang F., Tian P., Ma X. et al. EMU: Increasing the Performance and Applicability of LoRa through Chirp Emulation, Snipping, and Multiplexing // Proceedings. — 2022.
3. Mastronarde N., Russell D., Guan Z. RF-SITL: A Software-in-the-Loop Channel Emulator for UAV Swarm Networks. — 2022.
4. LoRa Alliance. LoRaWAN® Specification v1.0.4. — 2020.
5. Meshtastic documentation. — URL: <https://meshtastic.org/docs/introduction/> (дата обращения: 08.01.2026).
6. FutureSDR documentation. — URL: <https://www.futuresdr.org/learn/introduction.html> (дата обращения: 07.01.2026).

7. LoRa for FutureSDR — implementation overview. — URL: <https://www.futuresdr.org/blog/are-we-lora/> (дата обращения: 07.01.2026).
8. KISS Protocol Specification. — URL: https://m17-protocol-specification.readthedocs.io/en/latest/kiss_protocol.html (дата обращения: 08.01.2026).
9. Reticulum Network Stack Manual. — URL: <https://reticulum.network/manual/index.html> (дата обращения: 09.01.2026).